

解题报告

姓名：白家辉

班级：计算机大类2508

学号：8208250831

日期：2026.6.11

总览

本次解题报告中，共完成 4 道题目。

题目名称	难度	知识点
P2249 查找	普及	二分查找、lower_bound
P5076 普通二叉树	普及+/提高	二叉搜索树、排名查询、前驱后继
P3369 普通平衡树	提高+/省选-	平衡树、Treap、动态有序集合
P4305 不重复数字	普及	哈希、去重、顺序保留

（一）查找

题目信息

题目链接：<https://www.luogu.com.cn/problem/P2249>

知识点：二分查找、lower_bound

题目简述

输入一个单调不减序列，再给出若干次查询。对于每次查询，需要输出该数字在序列中第一次出现的位置；若不存在，则输出 -1 。

对于全部数据， $n \leq 10^6$ ， $m \leq 10^5$ 。

1.00s，128MB。

题目分析

1. 读题

题目只要求查询，不涉及修改，而且原数组已经按非降序排好。理论上二分查找非常自然，不过当前实现选择了另一条路线：直接把每个数第一次出现的位置存进哈希表，后续查询时做快速查找。

2. 性质观察

因为题目要求的是“第一次出现的位置”，所以无论使用什么方法，本质上都只需要记录某个数最早对应的下标。当前代码在读入有序序列时，若某个值第一次出现，就把它和位置一起插入哈希表；后续再遇到重复值则直接跳过。

3. 程序设计思路

1. 读入整个序列；
2. 对于每个数，如果它还没有出现在哈希表中，就把“数值 - 首次位置”这组信息插入哈希表；
3. 处理查询时，先检查哈希表中是否存在该值；
4. 若存在，就输出记录下来的首次位置；
5. 若不存在，则输出 -1。

4. 数据结构与算法选择

当前实现使用的是“拉链法哈希表”。每个哈希结点保存数值及其第一次出现的位置，因此查询时不再做二分，而是直接按值查表。

5. 时空复杂度分析

- 时间复杂度：平均情况下建表和查询都为 $O(1)$ ，总复杂度约为 $O(n + m)$ 。
- 空间复杂度： $O(n)$ ，需要为所有不同数值建立哈希结点。

洛谷 / 评测记录 / 评测详情

R281464384 记录详情

编程语言
C O2

代码长度
1.03KB

用时
1.87s

内存
16.97MB

测试点信息源代码

测试点信息

#1	#2	#3	#4	#5	#6
AC	AC	AC	AC	AC	AC
169ms/10.81MB	313ms/16.97MB	94ms/9.21MB	525ms/13.50MB	93ms/6.73MB	674ms/15.55MB

jsj0708Baijiahui

所属题目

P2249 【深基13.例1】查找

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 18:04:01

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
typedef struct Node
{
    int val;
    int firstpos;
    struct Node* next;
}Node;

#define hashsize 10003
Node* hashtable[hashsize];

int gethash(int x)
{
    return (x % hashsize);
}

int contains(int x)
{
    int pos = gethash(x);
    Node* p = hashtable[pos];
    while (p)
    {
        if (p->val == x)return 1;
        p = p->next;
    }
    return 0;
}

void insert(int x, int y)
{
    int pos = gethash(x);
    Node* p = (Node*)malloc(sizeof(Node));
    p->val = x;
    p->firstpos = y;
    p->next = hashtable[pos];
    hashtable[pos] = p;
}

int print(int x)
{
    int pos = gethash(x);

```

```

Node* p = hashtable[pos];
while (p)
{
    if (p->val == x)
        return (p->firstpos);
    p = p->next;
}
return 0;
}

int main()
{
    int n, m;
    scanf("%d %d", &n, &m);
    for (int i = 0; i < n; i++)
    {
        int num = 0;
        scanf(" %d", &num);
        if (!contains(num))
        {
            insert(num,i+1);
        }
    }
    for (int i = 0; i < m; i++)
    {
        int num = 0;
        scanf(" %d", &num);
        if (!contains(num))
        {
            printf("-1 ");
        }
        else
        {
            printf("%d ", print(num));
        }
    }
    return 0;
}

```

(二) 普通二叉树

题目信息

题目链接：<https://www.luogu.com.cn/problem/P5076>

知识点：二叉搜索树、排名查询、前驱后继

题目简述

需要维护一个数集，初始为空。支持插入一个互不重复的新数、查询某个数的排名、查询某个排名对应的数、查询前驱和后继。

与完整平衡树模板相比，本题没有删除操作，而且题目规模只有 10^4 级别。

对于全部数据，操作数 $q \leq 10^4$ 。

1.00s, 512MB。

题目分析

1. 读题

题目要求维护一个动态有序集合，并支持：

1. 排名查询；
2. 第 k 小查询；
3. 前驱、后继查询；
4. 插入操作。

这些操作都依赖“有序性”。不过当前代码并没有真的建一棵二叉搜索树，而是把已有元素始终维护在一个有序数组中，再配合二分查找完成各种查询。

2. 性质观察

由于没有删除操作，而且数据范围只有 10^4 级别，所以即使每次插入时挪动一段数组，整体也仍然可以接受。只要保证数组始终有序，就可以：

1. 用二分查找得到某个数应插入的位置，也就是它的排名；

- 2. 直接用下标访问第 k 小；
- 3. 利用相邻位置得到前驱和后继。

3. 程序设计思路

- 1. 用一个有序数组维护当前集合；
- 2. 通过 `binary_search` 找到某个值的 `lower_bound` 位置；
- 3. 查询排名时，直接返回这个位置；
- 4. 查询第 k 小时，直接访问数组第 k 个元素；
- 5. 查询前驱和后继时，也通过二分结果看相邻元素；
- 6. 插入时先找到应插入的位置，再把后面的元素整体向后移动一位。

4. 数据结构与算法选择

当前实现选择的是“有序数组 + 二分查找”。虽然从题意上它也可以归到有序集合维护问题，但对于本题这个数据规模来说，这种实现更短、更直接。

5. 时空复杂度分析

- 时间复杂度：查询排名、前驱、后继时二分部分是 $O(\log n)$ ，但插入需要整体搬移数组元素，最坏为 $O(n)$ ；在本题数据范围下依然可行。
- 空间复杂度： $O(n)$ ，只需要保存当前有序数组。

在本题给定的数据范围下，这样的实现通常可以接受。

代码实现

首页 / 评测记录 / 评测详情

R281464435 记录详情

编程语言C++2

代码长度1.12KB

用时28ms

内存1.04MB

测试点信息源代码

测试点信息

Subtask #0

#1AC4ms/788.00KB

#2AC4ms/788.00KB

#3AC4ms/860.00KB

#4AC4ms/788.00KB

#5AC5ms/788.00KB

Subtask #1

#6AC7ms/1.04MB

jsj0708Baijiahui

所属题目P5076 【深基16.例7】普通二叉...

评测状态Accepted

评测分数100

提交时间2026-06-11 18:04:30

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int arr[10001];
int size;

int binary_search(int x)
{
    int left = 1;
    int right = size;
    while (left <= right)
    {
        int mid = (left + right) / 2;
        if (arr[mid] >= x)
            right = mid - 1;
        else
            left = mid + 1;
    }
    return left;
}

int getrank(int x)
{
    return binary_search(x);
}

int getkth(int x)
{
    return arr[x];
}

int getprecursor(int x)
{
    int pos = binary_search(x);
    if (pos == 1)
    {
        return -2147483647;
    }
    return arr[pos-1];
}

int getsuccessor(int x)
{

```



```

    int pos = binary_search(x+1);

    if (pos > size)
    {
        return 2147483647;
    }
    return arr[pos];
}

void insert(int x)
{
    int pos = binary_search(x);
    for (int i = size; i >= pos; i--)
    {
        arr[i + 1] = arr[i];
    }
    arr[pos] = x;
    size++;
}

int main()
{
    int q = 0;
    scanf("%d", &q);
    while (q--)
    {
        int op, x;
        scanf("%d %d", &op, &x);
        switch (op)
        {
            case 1:
                printf("%d\n", getrank(x));
                break;
            case 2:
                printf("%d\n", getkth(x));
                break;
            case 3:
                printf("%d\n", getprecursor(x));
                break;
            case 4:
                printf("%d\n", getsuccessor(x));
                break;
            case 5:

```

```
        insert(x);
        break;
    }
}
return 0;
}
```

(三) 普通平衡树

题目信息

题目链接：<https://www.luogu.com.cn/problem/P3369>

知识点：平衡树、Treap、动态有序集合

题目简述

需要维护一个支持重复元素的动态有序集合，并支持插入、删除、查询排名、查询第 k 小、查询前驱、查询后继等操作。

对于全部数据，操作数可达到 10^5 ，因此必须保证每次操作都足够高效。

1.00s, 128MB。

题目分析

1. 读题

这是一道标准平衡树模板题。与前一题相比，本题不仅支持重复元素，还加入了删除操作，且数据范围更大，因此普通二叉搜索树已经不够稳妥。

2. 性质观察

为了保证所有操作都能在对数级别内完成，需要使用一种自平衡的数据结构，例如：

1. Treap;

2. Splay;
3. AVL;
4. 红黑树。

这些结构都能维护有序性，同时避免树退化成链。

3. 程序设计思路

以 Treap 为例，可以在每个结点中维护：

1. 关键字值；
2. 出现次数；
3. 子树大小；
4. 随机优先级。

之后：

1. 插入和删除通过旋转或 split/merge 维持平衡；
2. 排名查询根据左子树大小累计；
3. 第 k 小通过子树大小定位；
4. 前驱和后继通过沿搜索路径更新候选答案得到。

当前代码正是这种“数组模拟结点 + 随机优先级 + 左右旋”的经典 Treap 写法：insert 与 removeNode 负责维持平衡，zig、zag 用来完成旋转，pushup 负责更新子树大小。

4. 数据结构与算法选择

本题最典型的做法就是平衡树。当前实现使用的是普通旋转式 Treap，而不是 Splay 或 FHQ Treap，但核心目标一致，都是让树在随机意义下保持平衡。

5. 时空复杂度分析

- 时间复杂度：单次操作期望或均摊为 $O(\log n)$ 。
- 空间复杂度： $O(n)$ ，需要保存整棵平衡树结点信息。

这正是通过 10^5 次操作的关键。

洛谷 / 评测记录 / 评测详情

R281464482 记录详情

编程语言

C O2

代码长度

3.39KB

用时

187ms

内存

1.50MB

测试点信息

源代码

测试点信息

#1	AC	#2	AC	#3	AC	#4	AC	#5	AC	#6	AC	#7	AC
3ms/788.00KB		3ms/816.00KB		3ms/788.00KB		3ms/788.00KB		4ms/1.04MB		3ms/788.00KB		3ms/792.00KB	
#8	AC	#9	AC	#10	AC	#11	AC	#12	AC	#13	AC	#14	AC
6ms/788.00KB		19ms/1.04MB		40ms/1.33MB		37ms/1.39MB		38ms/1.42MB		3ms/816.00KB		22ms/1.50MB	

jsj0708Baijiahui

所属题目

P3369 【模板】普通平衡树

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 18:05:00

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAXN 100005

typedef struct {
    int val;
    int pri;
    int cnt;
    int siz;
    int l, r;
} Node;

Node tr[MAXN];
int tot = 0;
int root = 0;

int newNode(int val) {
    ++tot;
    tr[tot].val = val;
    tr[tot].pri = rand();
    tr[tot].cnt = 1;
    tr[tot].siz = 1;
    tr[tot].l = tr[tot].r = 0;
    return tot;
}

void pushup(int p) {
    tr[p].siz =
        tr[tr[p].l].siz +
        tr[tr[p].r].siz +
        tr[p].cnt;
}

void zig(int *p) {
    int q = tr[*p].l;
    tr[*p].l = tr[q].r;
    tr[q].r = *p;

    pushup(*p);
    pushup(q);
}

```

```

    *p = q;
}

void zag(int *p) {
    int q = tr[*p].r;
    tr[*p].r = tr[q].l;
    tr[q].l = *p;

    pushup(*p);
    pushup(q);

    *p = q;
}

void insert(int *p, int val) {
    if (!(*p)) {
        *p = newNode(val);
        return;
    }

    if (tr[*p].val == val) {
        tr[*p].cnt++;
    }
    else if (val < tr[*p].val) {
        insert(&tr[*p].l, val);

        if (tr[tr[*p].l].pri > tr[*p].pri)
            zig(p);
    }
    else {
        insert(&tr[*p].r, val);

        if (tr[tr[*p].r].pri > tr[*p].pri)
            zag(p);
    }

    pushup(*p);
}

void removeNode(int *p, int val) {
    if (!(*p)) return;

    if (tr[*p].val == val) {

```

```

    if (tr[*p].cnt > 1) {
        tr[*p].cnt--;
    }
    else if (!tr[*p].l || !tr[*p].r) {
        *p = tr[*p].l + tr[*p].r;
    }
    else if (tr[tr[*p].l].pri > tr[tr[*p].r].pri) {
        zig(p);
        removeNode(&tr[*p].r, val);
    }
    else {
        zag(p);
        removeNode(&tr[*p].l, val);
    }
}
else if (val < tr[*p].val) {
    removeNode(&tr[*p].l, val);
}
else {
    removeNode(&tr[*p].r, val);
}

if (*p) pushup(*p);
}

```

```

int getRank(int p, int val) {
    if (!p) return 1;

    if (val < tr[p].val)
        return getRank(tr[p].l, val);

    if (val > tr[p].val)
        return tr[tr[p].l].siz +
            tr[p].cnt +
            getRank(tr[p].r, val);

    return tr[tr[p].l].siz + 1;
}

```

```

int getKth(int p, int k) {
    if (!p) return 0;

```

```

    if (k <= tr[tr[p].l].siz)
        return getKth(tr[p].l, k);

    if (k <= tr[tr[p].l].siz + tr[p].cnt)
        return tr[p].val;

    return getKth(
        tr[p].r,
        k - tr[tr[p].l].siz - tr[p].cnt
    );
}

```

```

int getPre(int p, int val) {
    int ans = -2147483647;

    while (p) {
        if (tr[p].val < val) {
            ans = tr[p].val;
            p = tr[p].r;
        }
        else {
            p = tr[p].l;
        }
    }

    return ans;
}

```

```

int getSuc(int p, int val) {
    int ans = 2147483647;

    while (p) {
        if (tr[p].val > val) {
            ans = tr[p].val;
            p = tr[p].l;
        }
        else {
            p = tr[p].r;
        }
    }

    return ans;
}

```



```
int main() {
    srand(19260817);

    int n;
    scanf("%d", &n);

    while (n--) {
        int opt, x;
        scanf("%d%d", &opt, &x);

        if (opt == 1) {
            insert(&root, x);
        }
        else if (opt == 2) {
            removeNode(&root, x);
        }
        else if (opt == 3) {
            printf("%d\n", getRank(root, x));
        }
        else if (opt == 4) {
            printf("%d\n", getKth(root, x));
        }
        else if (opt == 5) {
            printf("%d\n", getPre(root, x));
        }
        else if (opt == 6) {
            printf("%d\n", getSuc(root, x));
        }
    }

    return 0;
}
```

(四) 不重复数字

题目信息

题目链接：<https://www.luogu.com.cn/problem/P4305>

知识点：哈希、去重、顺序保留

题目简述

题目有多组数据。对于每组输入的一列数字，要求删去其中重复出现的数，只保留每个数第一次出现时的那一次，并按原顺序输出。

也就是说，本题不是单纯把数字集合化，而是要在去重的同时保留首次出现顺序。

对于全部数据，单组长度可达到 5×10^4 。

1.50s, 128MB。

题目分析

1. 读题

题目要求做两件事：

1. 去重；
2. 保留第一次出现的顺序。

因此直接排序后去重并不符合要求，因为排序会破坏原本的出现顺序。

2. 性质观察

最自然的做法是顺序扫描整个序列，并用一个集合记录“这个数是否已经出现过”：

1. 如果没出现过，就输出并标记；
2. 如果已经出现过，就跳过。

这种方式既能去重，又不会改变原始先后顺序。

3. 程序设计思路

- 1. 先读入测试组数 τ ；
- 2. 对于每组数据，清空哈希集合；
- 3. 依次读入每个数；
- 4. 若当前数不在集合中，就把它加入集合并输出；
- 5. 若已存在，则直接忽略；
- 6. 每组数据处理结束后换行。

当前代码中用的是手写拉链法哈希表： `contains` 负责判断是否出现过， `insert` 把新数挂到对应桶链表头部， `clearhash` 则在每组数据开始前释放旧结点并清空哈希表。

4. 数据结构与算法选择

本题最适合使用哈希表或哈希集合。当前实现就是自定义链式哈希表，因此既能完成均摊常数时间判断，也能在多组数据之间显式重置状态。

5. 时空复杂度分析

- 时间复杂度：单组平均为 $O(n)$ 。
- 空间复杂度： $O(n)$ ，用于记录已经出现过的数。

在多组数据和较大值域下，哈希是比数组开桶更稳妥的选择。

代码实现

洛谷 / 评测记录 / 评测详情

R281464538 记录详情

编程语言C O2

代码长度1.22KB

用时2.41s

内存2.61MB

测试点信息源代码

测试点信息

#1 AC 8ms/1.02MB	#2 AC 7ms/1020.00KB	#3 AC 7ms/1020.00KB	#4 AC 68ms/1.25MB	#5 AC 69ms/1.38MB	#6 AC 68ms/1.25MB	#7 AC 564ms/2.50MB
#8 AC 528ms/2.50MB	#9 AC 546ms/2.49MB	#10 AC 541ms/2.61MB				

jsj0708Baijiahui

所属题目P4305 [JLOI2011] 不重复数字

评测状态Accepted

评测分数100

提交时间2026-06-11 18:05:32

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct Node
{
    int val;
    struct Node* next;
}Node;
#define hashsize 100003
Node* hashtable[hashsize];

int gethash(int x)
{
    return (x % hashsize + hashsize) % hashsize;
}
int contains(int x)
{
    int pos = gethash(x);
    Node* p = hashtable[pos];
    while (p)
    {
        if (p->val == x)return 1;
        p = p->next;
    }
    return 0;
}
void insert(int x)
{
    int pos = gethash(x);
    Node* p = (Node*)malloc(sizeof(Node));
    p->val = x;
    p->next = hashtable[pos];
    hashtable[pos] = p;
}
void clearhash()
{
    for (int i = 0; i < hashsize; i++)
    {
        Node* p = hashtable[i];
        while (p)
        {
            Node* tmp = p;
            p = p->next;

```

```

        free(tmp);
    }
    hashtable[i] = NULL;
}
}
int main()
{
    int T = 0;
    scanf("%d", &T);
    while (T--)
    {
        clearhash();
        int n = 0;
        scanf("%d", &n);
        for (int i = 0; i < n; i++)
        {
            int num = 0;
            scanf(" %d", &num);
            if (!contains(num))
            {
                insert(num);
                printf("%d ", num);
            }
        }
        printf("\n");
    }
    return 0;
}

```